

面试题总结

mmap映射的内存属于进程哪一块内存区域

mmap映射的区域位于用户空间的内存映射区，在堆和栈之间。这个区域向高地址方向增长，而栈在高地址向低地址增长，两者相向而行。

mmap分两个维度来理解：

第一个维度是映射对象：文件映射和匿名映射

文件映射有对应的磁盘文件，映射后内容就是文件内容，脏页会被回写到磁盘，主要用于高效读写文件和进程间共享文件。

匿名映射没有对应文件，映射后内容初始化为0，脏页不需要回写磁盘，主要用于分配大块内存或进程间共享内存。

代码上的区别是文件映射传入一个有效的文件描述符`fd >= 0`，匿名映射传入`fd = -1`并设置`MAP_ANONYMOUS`标志。

第二个维度是共享属性：`MAP_SHARED`和`MAP_PRIVATE`。

`MAP_SHARED`表示修改对其他映射了同一区域的进程可见，文件映射的修改回写到磁盘。`MAP_PRIVATE`表示写时复制（COW），进程修改时内核会复制一份私有副本，修改不影响源文件和其他进程。这两个维度可以组合出四种：共享文件映射、私有文件映射、共享匿名映射、私有匿名映射。

动态链接库（.so文件）就是通过mmap的共享文件映射加载到进程地址空间的，每个进程libc.so都映射到同一块物理内存，节省了大量物理页。

realloc函数的作用

realloc()函数的作用是重新调整已分配内存的大小，可以在原有基础上扩大或缩小。

函数原型是`void * realloc(void* ptr, size_t new_size)`，其中ptr是之前malloc/calloc/realloc返回的指针，new_size是新的请求大小。

缩小内存时，`realloc`直接截断，比如原来分配了100字节，现在`realloc`到40字节，那就把前40字节保留，后面的部分释放归还给堆管理器。返回的指针地址不变，还是原来的地址，数据的前40字节完好无损。

原地扩容时，如果原来分配的内存块后方有足够的空闲空间可以扩展，`realloc`会直接把后面的空闲空间纳入当前块，返回的指针地址不变。比如原来分配了40字节，后面恰好有40字节空闲，现在`realloc`到80字节，就直接往后扩展40字节，不需要移动数据，效率很高。

异地扩容时，如果后方空间不足以扩展，`realloc`会在堆中另外找一块满足`new_size`大小的连续空间，然后把旧数据完整拷贝过去，再自动释放旧空间，最后返回新地址。异地扩容的数据拷贝依赖的是`memcpy`或`memmove`函数，`glibc`内部实现用的是`memmove`，因为它能正确处理源地址和目标地址重叠的情况。

注意：如果`realloc`失败会返回`NULL`，但原来的内存还是有效的。如果直接写 `p = realloc(p, new_size)`，一旦失败原来的指针`p`就被`NULL`覆盖了，原来的内存再也找不到，导致内存泄漏。正确的做法是用一个临时指针接收返回值，判断非`NULL`之后再赋值给原指针。

什么情况会触发SIGSEGV段错误

`SIGSEGV`是操作系统发给进程的信号，表示进程试图访问非法的内存地址。触发场景有很多种：

- 解引用`NULL`指针。比如`int * p = NULL; *p = 42;`，地址0是不允许访问的
- 访问未映射的虚拟地址。随便给指针赋一个不存在的地址然后访问
- 访问已经释放的内存
- 栈溢出
- 写入只读内存区域。比如想修改常量区的变量，会触发`SIGSEGV`，因为试图写只读页
- 数组越界访问。
- 多次`free`同一个变量导致堆结构破坏后的异常访问
- 野指针访问。指针未初始化就使用
- 修改代码段。代码段`.text`是只读可执行的，试图往里面写数据会触发

堆溢出和栈溢出分别是什么？

栈溢出是指栈空间被耗尽。`Linux`默认给每个线程的栈大小是8MB，超出就会触碰栈底的保护页，触发`SIGSEGV`。

常见原因有三个：

- 递归调用太深
- 栈上分配超大局部变量
- 函数之间无限循环调用

堆溢出有两层含义。

第一是堆内存分配不足，比如不断malloc不释放，最终物理内存被耗尽，malloc返回NULL，如果不检查返回值继续使用就变成空间访问

第二是堆缓冲区溢出，也就是写入超出了malloc分配的边界。比如 `int* p = malloc(4)` 只分配了4字节却写 `p[100] = 42`，写到了远远超出边界的位置。

匿名管道读端关闭，写端写入会出现什么情况？

会触发SIGPIPE信号，这个信号的默认行为是终止进程。

常用的C++编译器有哪些？

GCC (g++) 是 GNU 编译器集合中的 C++ 编译器，Linux 系统的默认编译器，完全开源，跨平台支持 Linux、Windows (MinGW)、macOS 等，生态最成熟，标准支持最完善。

Clang 是 LLVM 项目的 C++ 编译器，编译速度比 GCC 快，错误和警告信息更友好易读，是 macOS 和 iOS 开发的默认编译器。Clang 的代码架构是库化的，方便做代码分析和重构工具。

MSVC 是微软的 Visual C++ 编译器，Windows 平台的原生编译器，集成在 Visual Studio 中，对 Windows API 和 COM 的支持最好，调试体验优秀。

Intel C++ Compiler (ICX) 是 Intel 的编译器，对 Intel CPU 的指令集优化做得很极致，高性能计算领域常用。新版 ICX 已经基于 LLVM 架构。

指针和引用的区别

指针和引用虽然都能间接操作另一个变量，但本质不同。

本质区别：指针是一个独立的变量，存储的是另一个变量的地址；引用不是一个独立变量，而是已有变量的别名，和原变量绑定在一起，没有自己的存储空间（编译器层面）。

初始化要求：指针可以不初始化，声明后指向随机地址（野指针）；引用必须在定义时初始化，绑定一个对象，不能先声明后绑定。

重新绑定：指针可以随时改变指向，从一个地址指向另一个地址；引用一旦绑定就终身不变，表面上给引用赋新值实际上是给原变量赋新值，不是重新绑定。

空值：指针可以为 `NULL` 或 `nullptr`，表示不指向任何对象；引用不存在"空引用"的概念，必须绑定到一个有效对象。

占用内存：指针自身占用内存，32 位系统上 4 字节，64 位系统上 8 字节，存储的是地址值；引用在语法层面不占额外空间，底层实现通常和指针一样传地址，但语言层面不允许对引用本身取 `sizeof`。

多级间接：指针支持多级，`int** pp` 是指向指针的指针，合法；引用不支持多级，`int&&` 在 C++11 中是右值引用，不是"引用的引用"。

运算：指针支持算术运算，`p++`、`p+n`、`p-p` 等；引用不支持任何算术运算。

使用建议：如果只需要修改值，不需要改指向，优先用引用，因为引用更安全（不可能为空、不可能悬空）、语法更简洁；如果需要改指向、可能为空、或者需要做指针算术，才用指针。

指针函数和函数指针的区别

指针函数是一个函数，它的返回值是指针类型。比如 `int * findMax(int*a,int *b)` 这个函数，返回类型是 `int *`，所以是指针函数。他就是一个普通的函数只不过返回值不是值，而是地址。

函数指针是一个指针，它指向的不是数据，而是一个函数。比如 `int (*funcPtr)(int, int)` 声明了一个函数指针，它可以指向任何签名为 `int(int, int)` 的函数。赋值时 `funcPtr = add` 就让它指向 `add` 函数，调用 `funcPtr(3, 5)` 就相当于调用 `add(3, 5)`。读法是"funcPtr 是一个指针，指向签名为 `int(int,int)` 的函数"。

函数参数传递的三种方式

值传递：函数接收的是实参的副本，在函数内部修改副本不影响原来的变量。比如 `void func(int x)`，调用 `func(a)` 时会把 `a` 的值拷贝一份给 `x`，修改 `x` 不会影响 `a`。对于基本类型（`int`、`double` 等）拷贝开销很小，值传递没问题；但对于大对象（比如一个很大的 `struct` 或 `string`），拷贝整个对象开销就很大了。

指针传递：函数接收的是实参的地址值，通过这个地址可以访问和修改原来的变量。比如 `void func(int* p)`，调用 `func(&a)` 时 `p` 存储的是 `a` 的地址，通过 `*p = 100` 就能修改 `a` 的值。指针传递只拷贝一个地址（4 或 8 字节），所以对大对象来说开销小很多。但指针可以为 `NULL`，使用前需要检查空指针；而且指针可以随时改指向别的对象。

引用传递：函数接收的是实参的别名，直接绑定到原变量上。比如 `void func(int& r)`，调用 `func(a)` 时 `r` 就是 `a` 的另一个名字，修改 `r` 就是修改 `a`。引用传递在底层实现上和指针传递一样传地址，但在语法上更简洁，不需要取地址和解引用的操作。引用必须绑定到一个有效对象，不存在空引用，所以比指针更安全。